Step-by-Step Testing Guide

This guide is written for another reader who wants to understand the benchmark project and reproduce or inspect the REST versus GraphQL load test results. It covers two workflows:

- Analyze the provided benchmark result archive without waiting for the full test suite.
- Run the benchmark from the APIs and generate a fresh result set.

Use the provided archive first if the goal is to review the existing benchmark data quickly. Run the full benchmark only when the goal is to reproduce the experiment on a new machine or after changing the code, database, or test configuration.

Project Testing Structure

The load testing files are organized as follows:

Path	Purpose
`tests/results.zip`	Provided benchmark results for immediate analysis
`tests/results/`	Directory where extracted or newly generated results are read from
`tests/k6/scenarios/`	REST and GraphQL k6 scenario scripts
`tests/k6/config/config.js`	Shared URLs, test users, load stages, and thresholds
`tests/k6/utils/`	Authentication and test data helpers
`tests/k6/run-all.sh`	Script that runs all REST and GraphQL scenarios as pairs
`scripts/monitor-resources.sh`	Docker resource monitoring script
`scripts/compare-results.js`	Script that compares REST and GraphQL result files

Fast Path: Analyze the Provided Results

The repository includes `tests/results.zip` so a reader can inspect the benchmark output without running all load tests. This is the recommended first step for reviewers, supervisors, or collaborators who only need to understand the results.

Only Node.js is required for this path. Docker, k6, the database, and running API containers are not required unless the reader wants to generate new results.

From the repository root:

```
mkdir -p tests/results
unzip -o tests/results.zip -d tests/results
node scripts/compare-results.js
```

The archive contains `rest/` and `graphql/` directories. After extraction, the result directory should look like this:

```
tests/results/
  rest/
    01-simple-list-summary.json
    01-simple-list-resources.csv
    ...
  graphql/
    01-simple-list-summary.json
    01-simple-list-resources.csv
    ...
```

If `tests/results/` already contains a result set that should be preserved, move it before extracting:

```
mv tests/results tests/results-local-backup
mkdir -p tests/results
unzip -o tests/results.zip -d tests/results
```

After running `node scripts/compare-results.js`, the terminal prints a scenario-by-scenario comparison of latency, throughput, error rate, CPU, memory, and transferred data.

Prerequisites

The full benchmark requires more setup than the fast path. Before running new tests, confirm that the following are available:

- Docker and Docker Compose are installed.
- The database service is available and seeded with the expected APS test data.
- Node.js dependencies are installed with `npm install`.
- k6 is installed and available from the terminal.
- The backend `.env` file is configured for the test environment.
- `DISABLE_OTP=true` is set before running k6 tests.

The k6 authentication helper logs in with phone number only and expects the login response to contain a JWT token. If `DISABLE_OTP` is not enabled, login returns an OTP session instead, and authenticated test scenarios will fail.

Verify the important environment variables:

```
grep -E "DISABLE_OTP|REST_PORT|GRAPHQL_PORT|JWT_SECRET|DB_" .env
```

The expected authentication setting is:

```
```

```
DISABLE_OTP=true
```

Step 1: Start and Verify the Backend

From the backend repository:

```
cd aps-be
docker-compose up -d --build
docker-compose ps
```

Expected services:

```
aps-rest-api       Up
aps-graphql-api    Up
```

If the database is managed by the separate `aps-db` project, verify that the database container is also running and reachable by both API containers.

Check the REST API health endpoint:

```
curl http://localhost:3001/health
```

Check the GraphQL server health endpoint:

```
curl http://localhost:3002/.well-known/apollo/server-health
```

Step 2: Confirm Login Behavior

The load tests require direct token login. Run these checks before the benchmark.

REST login:

```
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{ "phoneNumber": "+6281234567892" }'
```

The response must include `token` and `user`.

GraphQL login:

```
curl -X POST http://localhost:3002/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation Login($phoneNumber: String!) { login(input: {
phoneNumber: $phoneNumber }) { token user { id fullName role } } }",
    "variables": { "phoneNumber": "+6281234567892" }
  }'
```

The response must include `data.login.token`. If either API returns `sessionId` instead of `token`, update `.env`, set `DISABLE_OTP=true`, and recreate the API containers:

```
docker-compose up -d --force-recreate rest-api graphql-api
```

Step 3: Verify k6 and Scripts

Confirm k6 is installed:

```
k6 version
```

Make the benchmark scripts executable:

```
chmod +x tests/k6/run-all.sh
chmod +x scripts/monitor-resources.sh
chmod +x scripts/compare-results.js
```

The default k6 configuration is in:

```
tests/k6/config/config.js
```

The default endpoints are:

```
REST:    http://localhost:3001/api
GraphQL: http://localhost:3002/graphql
```

They can be overridden at runtime:

```
REST_URL=http://localhost:3001/api \
GRAPHQL_URL=http://localhost:3002/graphql \
k6 run tests/k6/scenarios/01-simple-list-rest.js
```

Step 4: Run a Smoke Test

Run one public REST scenario:

```
k6 run tests/k6/scenarios/01-simple-list-rest.js
```

Run the matching GraphQL scenario:

```
k6 run tests/k6/scenarios/01-simple-list-graphql.js
```

A successful smoke test should show:

- `http_req_failed` below the configured threshold.
- `checks` above the configured threshold.
- HTTP status checks passing.
- No authentication errors for scenarios that require tokens.

If Scenario 1 passes but Scenario 2 fails with authentication errors, recheck `DISABLE_OTP=true` and repeat the login verification step.

Step 5: Run the Full Paired Benchmark

Run all scenarios:

```
./tests/k6/run-all.sh
```

The script runs each scenario as a REST and GraphQL pair. It also starts Docker resource monitoring for the API container under test.

Current paired scenarios:

Scenario	Name	Main comparison
01	Simple list	Public facility list retrieval
02	List with relations	REST relational endpoint vs GraphQL nested selection
03	Filtered slots	Parameterized availability query
04	Generic nested query	Client-composed REST calls vs GraphQL composed query
05	Optimized nested query	Dedicated dashboard endpoint/resolver
06	Booking creation	Write operation and conflict handling

The full run usually takes more than one hour because each scenario runs for approximately five minutes per API, with cooldown periods between pairs.

During the benchmark:

- Keep the machine workload stable.
- Do not restart containers.
- Do not modify code, database seed data, or Docker resource limits.
- Keep the terminal open until the script finishes.
- Record the date, machine specifications, Docker limits, and git revision.

Step 6: Monitor Resources Manually, If Needed

The full runner starts resource monitoring automatically. For manual observation in another terminal:

```
docker stats aps-rest-api aps-graphql-api
```

For a separate CSV capture:

```
./scripts/monitor-resources.sh 300 tests/results/manual-resources.csv all
```

The monitor records CPU, memory, and network statistics every five seconds.

Step 7: Analyze Results

After extracting `tests/results.zip` or completing a fresh benchmark run:

```
node scripts/compare-results.js
```

The comparison script reads:

```
tests/results/rest/*-summary.json
tests/results/graphql/*-summary.json
tests/results/rest/*-resources.csv
tests/results/graphql/*-resources.csv
```

Important metrics to extract:

- Average, median, p90, and p95 latency.
- Requests per second.
- Total requests.
- Error rate.
- CPU usage.

- Memory usage.
- Data received and sent.

Example direct extraction:

```
jq '.metrics.http_req_duration.values["p(95)"]' tests/results/rest/01-
simple-list-summary.json
jq '.metrics.http_reqs.rate' tests/results/rest/01-simple-list-summary.json
jq '.metrics.http_req_failed.rate' tests/results/rest/01-simple-list-
summary.json
```

Step 8: Organize Result Files

The comparison script expects this structure, whether the files came from `tests/results.zip` or a fresh run:

```
tests/results/
  rest/
    01-simple-list-raw.json
    01-simple-list-summary.json
    01-simple-list-resources.csv
    ...
  graphql/
    01-simple-list-raw.json
    01-simple-list-summary.json
    01-simple-list-resources.csv
    ...
```

Keep the raw JSON and CSV files. They provide the evidence needed to reproduce tables, charts, and statistical analysis.

For distributed documentation, include `tests/results.zip` when sharing the project. It allows another person to verify the comparison output without repeating the long-running benchmark.

For formal reporting, create a run log with:

- Date and time of the run.
- Host CPU, memory, and operating system.
- Docker resource limits.
- Backend git commit.
- Database seed version or dataset description.
- Value of `DISABLE_OTP`.
- k6 version.
- Notes about unexpected errors or interruptions.

Troubleshooting

k6 is not found

Install k6 and verify the installation:

```
k6 version
```

Login returns `sessionId` instead of `token`

The OTP flow is still enabled. Set:

```
DISABLE_OTP=true
```

Then recreate both API containers:

```
docker-compose up -d --force-recreate rest-api graphql-api
```

Repeat the REST and GraphQL login checks before rerunning k6.

Authenticated scenarios fail with 401

Possible causes:

- `DISABLE_OTP` is not enabled in the running container.
- `JWT_SECRET` differs between token creation and token verification.
- The test phone numbers do not exist in the seeded database.
- The API containers were not recreated after `.env` changes.

Check the effective container environment:

```
docker exec aps-rest-api printenv DISABLE_OTP
docker exec aps-graphql-api printenv DISABLE_OTP
```

Services are not healthy

Review container status and logs:

```
docker-compose ps
docker-compose logs rest-api
docker-compose logs graphql-api
```

Restart the services if needed:

```
docker-compose restart rest-api graphql-api
```

Booking creation has conflicts

Some booking conflicts are expected in Scenario 06 because concurrent users may attempt to create bookings for overlapping facilities and times. Treat conflict rates as part of the write workload analysis, but investigate them if they exceed the configured threshold or are accompanied by system failures.

Results vary between runs

Benchmark variation is normal. For thesis reporting:

- Run each scenario set multiple times.
- Use the median result or report all runs transparently.
- Avoid heavy background applications.
- Keep Docker resource limits unchanged.
- Allow cooldown time between runs.

Research Reporting Checklist

Before using the results in a thesis or report, confirm that:

- The same backend commit was used for both APIs.
- REST and GraphQL used the same database and seed data.
- `DISABLE_OTP=true` was enabled for both API containers.
- Both APIs used equivalent Docker resource limits.
- The full scenario set completed without interruption.
- Raw result files were preserved.
- Any excluded run is documented with a reason.

The final report should include tables for latency, throughput, error rate, and resource utilization, followed by a short interpretation of the observed differences for each scenario.